

# REEMAN SLAM WEB APIs

---

Navigation version:RSN-v3.1.5

Creation date:2023-05-8

Date Modified:2023-05-8

---

## Glossary:

**host:belowhost**Indicates the machine's navigation system'sIPaddress,IPThe third digit of the network segment cannot be10network segment, such as:192.168.10.123is an illegal address, please change it to another non-10Network segment router or change routerwifinetwork segment.127.0.0.1expressThe machine is not connected to any network, please connect to the network first. When calling the interface, the calling end needs to be connected to the same LAN as the navigation system. When using the interface, please set thehostreplace with the correspondingIPaddress.

**hostname:**Refers to the name of the navigation host

**Navigation mode:** Navigation mode means that the machine can execute navigation instructions in this mode, corresponding to the mapping

**modeMapping mode:** Mapping mode means that the machine performs the task of scanning maps, and the scanning of maps can be completed in this

**mode.QR:** Used for auxiliary positioningQR code, some models come with

**virtual wall:**Can be added on the map to limit the range of movement of the machine

**Special area:** A special area can be added on the map to complete certain functions, such as changing the speed of the machine in the special area. machine entryThe special area will be reported to the host computer through the serial port.

## Basic Information

### Get the current navigation version

```
-method: GET
-path: http://(host)/reeman/current_version
-response:
  -Content-Type: application/json
  -result: {"version": "v1.1.0"}
-mark:Returns the current version name
```

## Obtain robot pose information

```
-method: GET
-path: http://(host)/reeman/pose
-response:
  -Content-Type: application/json
  -result: {"x":297,"y":251,"theta":0.97}
  -mark:Return the current coordinates of the machine
```

## Get the current mode

```
-method: GET
-path: http://(host)/reeman/get_mode
-response:
  -Content-Type: application/json
  -result: {"mode":2}
  -mark:mode=1for the construction mode;mode=2for navigation mode
```

## get machinehostname

```
-method: GET
-path: http://(host)/reeman/hostname
-response:
  -Content-Type: application/json
  -result: {"hostname":"reeman-robot"}
  -mark:return machinehostname
```

## ObtainIMUstate

```
-method: GET
-path: http://(host)/reeman/imu
-response:
  -Content-Type: application/json
  -result: {"a":0.0,"g":0.0}
  -mark:returnresultThe result is normal, if returnexceptionis abnormal.
```

## Acquire Laser Data

```
-method: GET
-path: http://(host)/reeman/laser
-response:
  -Content-Type: application/json
  -result: {"coordinates": [[364,166], [365,167], [
321, ...]]}
  -mark:Back to laser dot matrixx,y2D array of coordinates
```

## Get power management status

```
-method: GET
-path: http://(host)/reeman/base_encode
-response:
  -Content-Type: application/json
  -result: {"battery":100,"chargeFlag":1,"emergencyButton":0}
  -mark: in, battery for electricity (0-100); chargeFlag For charging status: 2-Charging at the
    charging pile; 3-Adapter charging
    electricity; 8-The charging pile is being docked; other values indicate a non-charging
    state; emergencyButton For emergency stop switch state: 0 for pressing, 1 for bounce.
```

## get current speed

```
-method: GET
-path: http://(host)/reeman/speed
-response:
  -Content-Type: application/json
  -result: {"vx":0.3,"vth":0.5}
  -mark: vx is the linear velocity, the unit is m/s; vth is the angular velocity, in
    degrees/s
```

## reset

```
-method: POST
-path: http://(host)/cmd/reloc_pose
-body: {"x":1.0,"y":3,"theta":0.14} //x,y,theta are the coordinates and direction of the
relocation
-response:
  -Content-Type: application/json
  -result: {"status":"success"}
```

## navigation module

### Send location coordinates for navigation

```
-method: POST
-path: http://(host)/cmd/nav
-body: {"x":285,"y":252,"theta":1.6}
-response:
  -Content-Type: application/json
  -result: {"status":"success"}
  -mark: theta in radians
```

### Send destination name navigation

```
-method: POST
-path: http://(host)/cmd/nav_name
-body: {"point":"A"}
-response:
  -Content-Type: application/json
  -result: {"status":"success"}
  -mark: Alt is the name of the target point, and a normal return means that the target point is
    found, otherwise return Exception
```

## cancel navigation

```
-method: POST
-path: http://(host)/cmd/cancel_goal
-body: {}
-response:
  -Content-Type: application/json
  -result: {"status": "success"}
-mark: Normal return indicates success, otherwise return Exception
```

## navigation charging

```
-method: POST
-path: http://(host)/cmd/charge
-body: {"type": 0, "point": "Charging pile"}
  -mark: type=0 Indicates that it is directly connected to the charging
    pile; type=1 Indicates navigating to the location of the charging pile and starting to dock
    the charging pile, "Charging pile" bit
The setting needs to be calibrated on the web page first
-response:
  -Content-Type: application/json
  -result: {"status": "success"}
-mark: Normal return indicates success, otherwise return Exception
```

## Send multiple target coordinates for navigation

```
-method: POST
-path: http://(host)/cmd/nav_list
-body: [{"name1": ["x", "y", "theta"]}, {"name2": ["x", "y", "theta"]}]]
  -in name1 is the target point name; x, the y, theta corresponding to the coordinates x, the
    y coordinates and angles
-response:
  -Content-Type: application/json
  -result: {"status": "success"}
-mark: theta in radians
```

## Set navigation maximum speed

```
-method: POST
-path: http://(host)/cmd/max_speed
-body: {"speed": 0.5}
-response:
  -Content-Type: application/json
  -result: {"status": "success"}
-mark: Velocity values range from 0.3 to 1.0 m/s
```

# map module

## get current map

Get the current map data, including map size and other data, see the returned results for details.

```
-method: GET
-path: http://(host)/reeman/map
-the body: {}
-response:
  -Content-Type: application/json
  -result: {}
```

## switch drawing mode

After switching to the mapping mode, the machine will perform mapping scanning, please push or remote control the robot to move and scan. After the scan is complete click BuildFinish building the map.

```
-method: POST
-path: http://(host)/cmd/set_mode
-the body: {"mode":1}
-response:
  -Content-Type: application/json
  -result: {}
```

## Toggle incremental mapping mode

Switch to the incremental mapping mode. Please ensure that the positioning is correct before switching, otherwise the map may deviate from the actual environment.

```
-method: POST
-path: http://(host)/cmd/set_mode
-the body: {"mode":3}
-response:
  -Content-Type: application/json
  -result: {}
```

## save map

Save the current scanned map. If you want to give up this map and switch to the navigation mode, you need to switch to the navigation mode after saving the map. Navigate normally.

```
-method: POST
-path: http://(host)/cmd/save_map
-the body: {}
-response:
  -Content-Type: application/json
  -result: {}
```

## Get the name of the current map

Returns the name of the map used by the current navigation system

```
-method: GET
-path: http://(host)/reeman/current_map
-response:
  -Content-Type: application/json
  -result: {"alias":"","name":"674e32391fad7ad8a1422360357d0516"}
  -mark:inaliasesis an alias for the map, nameuniquely identifies the map
```

## Get a list of all maps

Returns a list of all maps in the current navigation system

```
-method: GET
-path: http://(host)/reeman/history_map
-response:
  -Content-Type: application/json
  -result: {map list}
```

## application map

Switch to the specified map. After switching the map, it will relocate at the location of the charging pile by default. It is recommended to switch the map at the location of the calibrated charging pile.

```
-method: POST
-path: http://(host)/cmd/apply_map
-the body: {"name":"map name"}
-response:
  -Content-Type: application/json
  -result: {}
```

## export map

Download the specified map to the local

```
-method: POST
-path: http://(host)/download/export_map
-the body: {"name":"map name"}
-config:{responseType:"blob"}
-response:
  -Content-Type: application/json
  -result:
```

## upload map

Select the map file to upload to the navigation system, the body: `let data=new FormData(); data.append('file',file)`

```
-method: POST
-path: http://(host)/upload/import_map
-the body:{data}
-response:
  -Content-Type: application/json
  -result: {}
```

## rename map

Please be careful when renaming map names.

```
-method: POST
-path: http://(host)/cmd/rename_map
-body: {"old_name":"old name","new_name":"new name"}
-response:
  -Content-Type: application/json
  -result: {}
```

## delete map

Delete the map with the specified name, it cannot be restored after deletion, please operate with caution

```
-method: POST
-path: http://(host)/cmd/delete_map
-body: {"name":"map name"}
-response:
  -Content-Type: application/json
  -result: {}
```

## Get calibration position

Get a list of calibrated coordinate positions

```
-method: GET
-path: http://(host)/reeman/position
-the body:{}
-response:
  -Content-Type: application/json
  -result: {"waypoints":[{"name":"1","type":"delivery","pose":
{"x":-
2.06,"y":0.77,"theta":0.19}},{ "name":"3","type":"avoid","pose":{"x":-
0.91,"y":0.48,"theta":-0.01}}]}
```

-mark:Returns a list of calibration locations, where **name** Indicates the location name of the calibration, **type** Refers to the location type, **pose** for the location coordinates

## get special area

get special area

```
-method: GET
-path: http://(host)/reeman/special_polygon
-the body:{}
-response:
  -Content-Type: application/json
  -result: {"polygons":[{"name":"","polygon":[[],[]],"speed":0.6,"type":0},
{}...]}
```

## virtual wall

### get virtual wall

```
-method: GET
-path: http://(host)/reeman/restrict_layer
-body:{}
-response:
  -Content-Type: application/json
  -result: Returns a two-dimensional array of virtual wall points
```

## update virtual wall

```
-method: POST
-path: http://(host)/cmd/restrict_layer
-body: {"waypoints":[[1,1],[x,y]...]} //Two-dimensional array
-response:
  -Content-Type: application/json
  -result: {}
```

## move

Manually control the movement of the robot

```
-method: POST
-path: http://localhost/cmd/speed
-body: {"vx":Straight distance-Unit: m,"vth":rotation angle-unit radian}
-response:
  -Content-Type: application/json
  -result: {}

-mark:1.Forward: the forward straight parameter
is a positive number, and the angle
parameter is0 as forward1rice:the
body={"vx":1,"vth":0}

2.Reverse: the parameter of the
straight line is negative, and the
parameter of angle is0 as back1:the
body={"vx":-1,"vth":0}

3.Turn left: go straight and the parameter is0, the angle parameter is a positive number
turn left0.5radian:the body={"vx":0,"vth":0.5}

4.Turn right: go straight with parameter0, the angle parameter is negative
if turn right0.5radian:the body={"vx":0,"vth":-0.5}

5.Stop: go straight, turn parameters are0
```



# Liftclass

## Upgraded navigation system

Select the upgrade file to upload to the navigation system for upgrade, and it will automatically upgrade after the upload is successful.

the body:let data=new FormData(); data.append('file',file)

```
-method: POST
-path: http://(host)/upload/upgrade
-the body:{data}
-response:
  -Content-Type: application/json
  -result: {}
```

## External Power Control

Control external power switch

Turn on the power

```
-method: POST
-path: http://localhost/cmd/lock
-body: {}
-response:
  -Content-Type: application/json
  -result: {}
```

power off

```
-method: POST
-path: http://localhost/cmd/unlock
-body: {}
-response:
  -Content-Type: application/json
  -result: {}
```